

Tema 4 : Entrada - Salida

Necesidad de la E/S

- ↳ Entrada de información para escribir en la memoria desde el exterior
- ↳ salida de información para conocer los resultados de la ejecución de los programas
- ↳ Interacción CPU y Memoria con el mundo (dispositivo)

El rendimiento del sistema de E/S determina el rendimiento global del sistema

Tipos de periféricos dispositivos que permiten la entrada y salida de datos autónomos

- Presentación de datos interaccionan con los usuarios, transportando datos entre estos y la máquina
Ratón, teclado, pantalla, impresora,...
- Comunicación con otros procesadores remotos a través de redes
Tarjeta de red, módem,...
- Adquisición de datos comunicación con sensores y actuadores que operan de forma autónoma
se utilizan en sistemas de control automático de procesos por computador
suelen incorporar conversores de señales A/D y D/A
- Almacenamiento de datos forman parte de la jerarquía de memoria: interacción de forma autónoma con la máquina
Discos magnéticos y cintas magnéticas...

Tipo de datos que se transmiten: bytes, bloques, tramas, etc.

Tasa de transferencia

- Ancho de banda cantidad de datos que se pueden transferir por unidad de tiempo
(Mbit/s, MB/s, ...) Dispositivos de almacenamiento/red → gran ancho de banda
- Latencia tiempo requerido para obtener el primer dato (s, ms, μ s, ns)

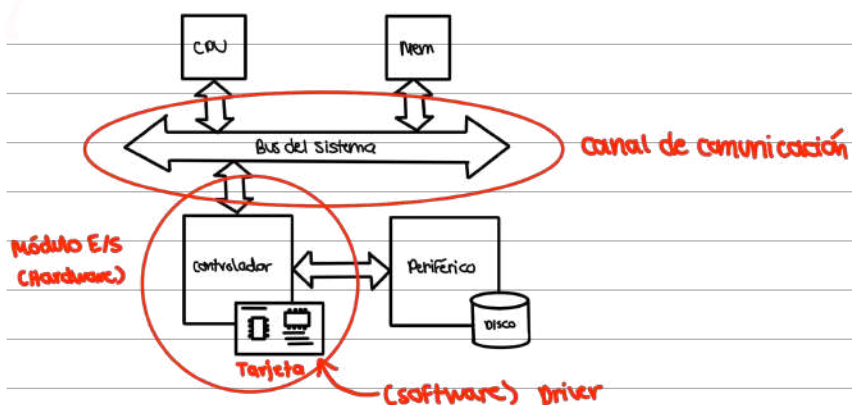
Clasificación de los dispositivos de E/S

Comportamiento entrada, salida, almacenamiento, comunicación

Destinatario humano o máquina

Tasa de referencia bytes/seg, transfers/seg

Teclado	Entrada	Humano	0,01 kB/s
Impresora Láser	Salida	Humano	100 kB/s
Red LAN	comunicación	Máquina	10000 kB/s



Componentes

Dispositivo periférico

Controlador hardware interfaz lóg. conecta con el bus y conoce fís. dispositivo

Controlador software código encargado de programar el cont. HW driver

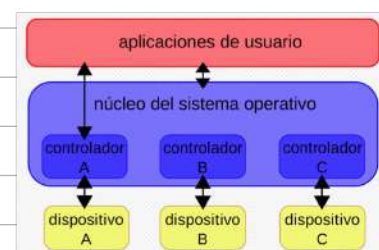
Canal de comunicación

interconexión CPU, memoria y controlador HW

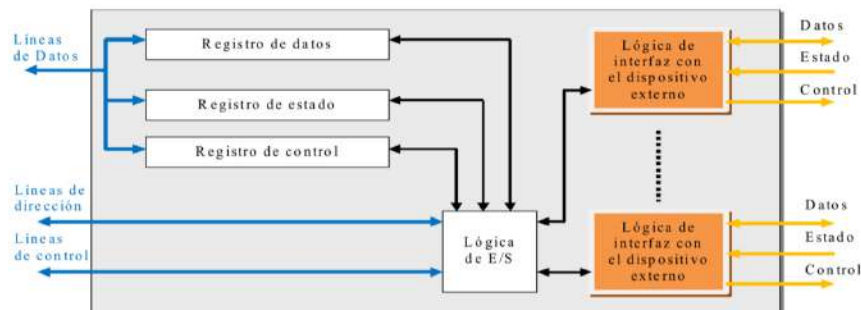
Controlador del dispositivo

funciones:

- controlador y temporización : adapta la velocidad de transferencia
- comunicación con el procesador } 2 interfaces
- comunicación con el periférico
- Buffering o almacenamiento intermedio
- Detección de errores



ESTRUCTURA DE UN CONTROLADOR E/S



- Registro de datos: intercambio de datos entre CPU y periférico
- Registro de estado: estado del dispositivo, ej. preparado, error, ocupado...
- Registro de control: actuación sobre el dispositivo (imprime un carácter, lee un bloque...)

CUIDADO: NO son registros de la CPU. ¡¡¡Pueden estar en otro chip!!!

Operación de E/S

Petición envía comando y dirección. CPU indica al dispositivo la operación que quiere hacer escribiendo en su registro de control

¿cómo seleccionar dispositivo E/S? depende de que sea aislada o mapeada en memoria

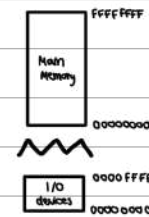
Envío transferencia de los datos. CPU leerá/ escribirá de/al registro de datos del controlador del periférico deseado

¿cómo saber cuándo? SINCRONIZACIÓN

Direccionamiento de controladores métodos de implementación de E/S entre periféricos y CPU

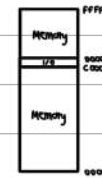
E/S aislada espacio de direcciones diferente al de la memoria principal (dos mapas de direcciones)

Existen instrucciones específicas de E/S (IN, OUT de Intel)



E/S mapeada en memoria memoria y dispositivos E/S comparten el espacio de direcciones

Se puede usar instrucciones tipo load/store



Sincronización exigida por la diferencia de velocidad entre CPU y dispositivos de E/S

Mecanismos de sincronización

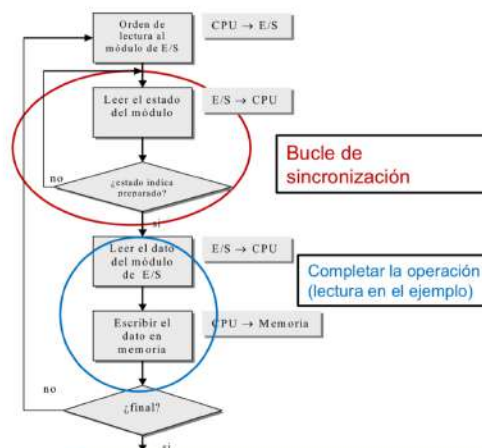
E/S programada con espera de respuesta (polling) la más sencilla de implementar, pero presenta inconveniente de pérdida de tiempo (no realiza trabajo útil en el bucle de espera)

E/S por interrupciones aprovecha mejor el tiempo de CPU, permitiendo ejecución concurrente

E/S programada

OPERACIÓN DE E/S PROGRAMADA

- ⊙ Cada vez que la CPU quiere realizar una transferencia:
 - ⊙ Lee una y otra vez el **registro de estado** del periférico (encuesta o "polling") hasta que esté preparado para realizar la transferencia
 - ⊙ Realiza la **transferencia**

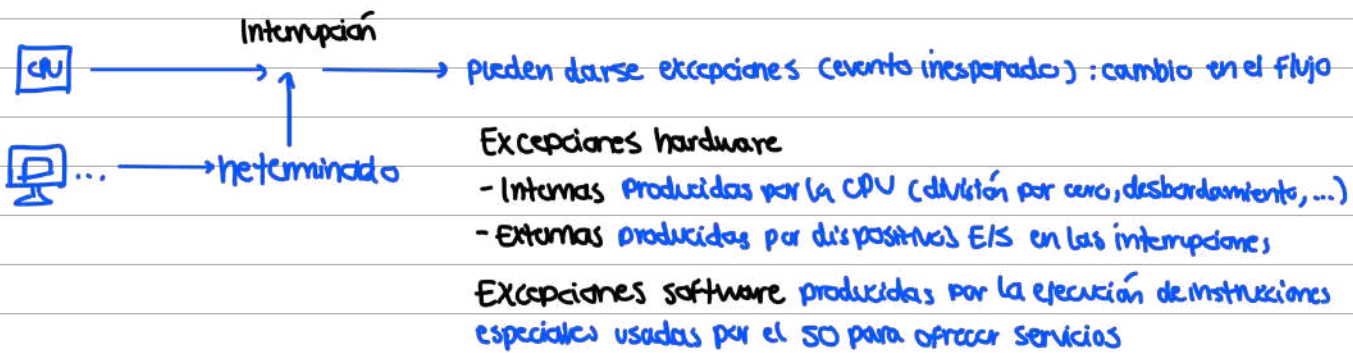


Solución parcial: fijar un límite de tiempo en el bucle de espera

PROBLEMAS DE LA E/S PROGRAMADA

- ⊙ La CPU no hace trabajo útil en el bucle de espera activa
 - ⊙ Con dispositivos lentos el bucle podría repetirse miles/millones de veces
 - ⊙ La dinámica del programa se detiene durante la operación de E/S
 - Ejemplo: imaginar que en un videojuego se detuviese la dinámica del juego a la espera de que el usuario pulse una tecla
- ⊙ Dificultades para atender a varios periféricos
 - ⊙ Mientras se espera a que un periférico esté listo para transmitir, no se puede atender a otro

E/S por interrupciones dispositivo avisa a la CPU cuando termina su operación y permite que la CPU haga trabajo útil mientras el periférico hace la operación solicitada. La CPU solo tendría que retomar el control de la operación cuando el periférico hubiese terminado



- 1) CPU interrumpe la ejecución de instrucciones
- 2) Guarda información para poder retomar ejecución
- 3) Cambia a un estado especial para tratar la excepción
- 4) Pasa a ejecutar Rutina de Tratamiento de Interrupción / Excepción (RTI)
- 5) Retorno de la RTI retomaría ejecución del programa original (o no)

Subrutina vs RTI

Similitudes

Se rompe secuencia normal de ejecución

Cuando terminan de ejecutarse se debe retornar al punto de ruptura (por eso el HW guarda automáticamente el PC)

RTE (instrucción de retorno de excepción)

Diferencias

Subrutina: el programador sabe en qué punto exacto se rompe la secuencia

RTI: puede ejecutarse en cualquier momento sin control del programador.

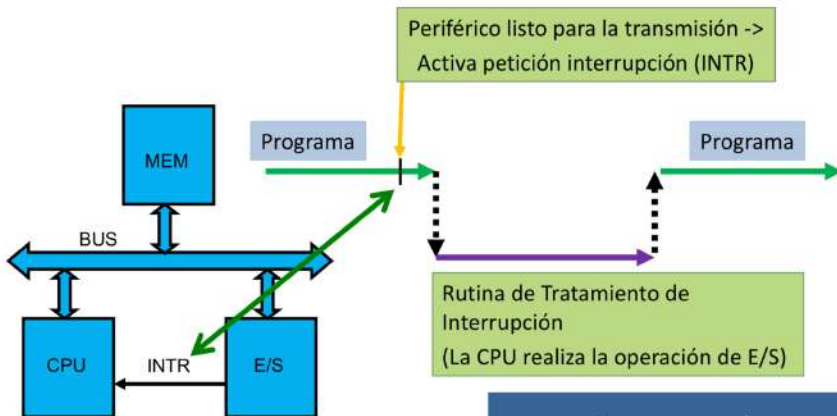
Es necesario guardar el registro de estados y todos los registros que usa y restaurarlos al retornar

Excepción vs Interrupción

Excepción interna: ejecución incorrecta de instrucciones del programa → procesador bifurca a la subrutina asociada a la excepción

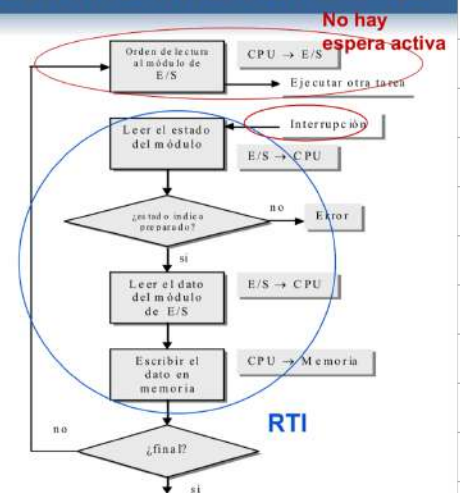
Interrupción: señal externa al procesador. Evento asíncrono que avisa al procesador de que necesita su atención

OPERACIÓN DE E/S MEDIANTE INTERRUPCIONES



OPERACIÓN DE E/S MEDIANTE INTERRUPCIONES

- La CPU hace la **petición de operación de E/S** y pasa a ejecutar otros programas. => No existe bucle de espera
- Cuando un periférico está listo para transmitir se lo indica a la CPU activando una **LÍNEA DE PETICIÓN INTERRUPTIÓN**
- Cuando la CPU recibe una señal de petición de interrupción salta a una **RUTINA DE TRATAMIENTO DE INTERRUPTIONES (RTI)**, que se encarga de atender al periférico que interrumpió y **realizar la operación de E/S**



Por ejemplo: Quiero ver una imagen

- CPU le pide a la pantalla de que renderice la imagen y pasa a ejecutar otros programas
- La pantalla está lista y le avisa a la CPU activando una interrupción por petición
- CPU recibe aviso, salta la RTI para atender a la pantalla y realizar la operación de Salida

LA GESTIÓN DE INTERRUPTIONES

- El programa **activa** el sistema de interrupciones
- Periférico activa **petición interrupción (INTR)**
- CPU termina de ejecutar la **instrucción en curso** y **comprueba si hay interrupciones pendientes**
- CPU **salva automat. contexto** (PC y reg. de estado), **desactiva las interrupciones** y **salta a la RTI**
- CPU ejecuta la **RTI**, durante la cual realiza la operación de E/S
- CPU **capacita las interrupciones**, **restaura contexto** (PC y reg. estado) y **retorna al programa**
- CPU continúa con la ejecución del programa

HABILITACIÓN DE INTERRUPCIONES

- 1 El programa **activa** el sistema de interrupciones

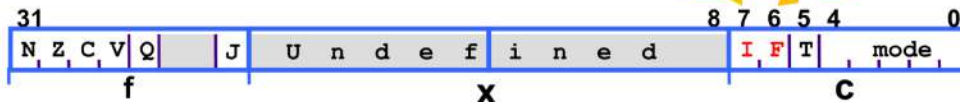


Para que pueda producirse una interrupción la CPU debe permitir que un dispositivo le interrumpa, **habilitando las interrupciones**:

Localmente: se indica al controlador del dispositivo que puede generar una señal de interrupción

Globalmente: en el registro de estado de la CPU se activa el bit correspondiente a la línea de interrupción

IRQ FIQ

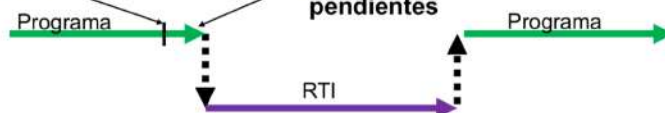


IRQ interrupt request
(global)

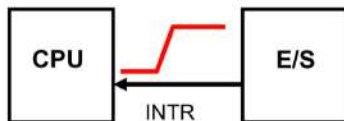
FIQ fast interrupt request
(alta prioridad)

COMPROBACIÓN DE PETICIÓN DE INTERRUPCIÓN

- 2 Periférico activa **petición interrupción** (INTR)



- 3 CPU termina de ejecutar la **instrucción en curso** y **comprueba si hay interrupciones pendientes**



COMPROBACIÓN DE PETICIÓN DE INTERRUPCIÓN

- La CPU **comprueba si hay interrupciones pendientes** (línea INTR activada) al final de la ejecución de cada instrucción

Motivo:

- Sólo es necesario guardar el PC, el registro de estado y los registros accesibles por programa (registros de datos y/o direcciones)
- Si se interrumpiese una instrucción en mitad de la ejecución sería necesario guardar el valor de todos los registros internos => estado completo de la CPU
 - Registro de instrucción, registros de dirección de datos, registros de datos de memoria, etc.

Salvo para:

- Instrucciones de larga duración
 - Por ejemplo, en instrucción de movimiento múltiple (STM), se comprueba si hay interrupciones pendientes después de mover cada una de las palabras

SALTO A LA RTI



- 4 CPU salva automáticamente el contexto (PC y registro de estado), deshabilita las interrupciones y salta a la RTI

Salvar el estado:

El procesador guarda **automáticamente** el contexto del programa en ejecución (En el ARM el PC y registro de estado) en una zona de la pila distinta de la del programa o en registros especiales

Deshabilitar (enmascarar) las interrupciones:

En el registro de estado se enmascaran (deshabilitan) **automáticamente** las interrupciones por la línea INTR

¿Por qué?

Saltar a RTI:

Se obtiene la dirección de la RTI que corresponda al periférico que ha interrumpido

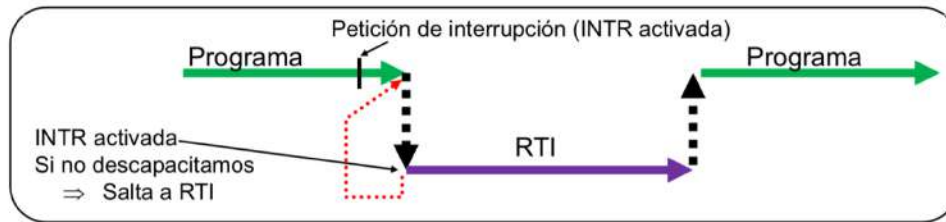
¿Cómo se identifica al periférico?

DESHABILITAR LAS INTERRUPCIONES

Antes de saltar a la RTI es necesario enmascarar las interrupciones

Motivo: Si no se enmascaran la CPU puede entrar en un bucle infinito

- Cuando se entra en la RTI el periférico todavía no ha desactivado su petición
- Si las interrupciones están habilitadas $\Rightarrow$ la CPU detecta una interrupción pendiente y vuelve a saltar a la RTI una y otra vez
- Antes de finalizar la RTI hay que asegurarse de que el periférico ha desactivado la línea de petición INTR



Alternativas

- **Enmascaramiento global**
 - Se deshabilitan todas las interrupciones $\Rightarrow$ ningún otro periférico podrá interrumpir durante la ejecución de la RTI
- **Enmascaramiento selectivo**
 - Cuando hay varios niveles de interrupción se pueden deshabilitar las interrupciones por el nivel que interrumpe, pero no necesariamente por el resto de niveles. Ej: IRQ y FIQ

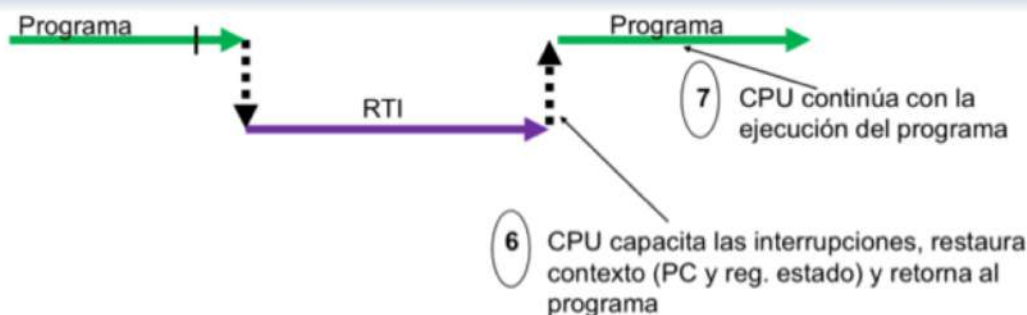
EJECUCIÓN DE LA RTI



Responsabilidades de la RTI:

- Salvar en pila todos los registros que utiliza (manual)
 - Si va a invocar a subrutinas, tiene que cumplir el estándar de subrutinas
- Informar al periférico de que se ha reconocido su interrupción. ¿Cómo?
 - Por **software**: borrar flag de petición en un registro de control
 - Por **hardware**: activando una señal de reconocimiento de interrupción (INTA)
- **Una vez informado el periférico desactiva INTR**
- Realizar la operación de E/S con el periférico
- Restaurar el contexto arquitectónico salvado
- Realizar el retorno de interrupción

AL FINAL DE LA EJECUCIÓN DE LA RTI



GESTIÓN DE INTERRUPCIONES

1. El programa activa el sistema de interrupciones

2. Periférico activa petición interrupción (INTR)

3. CPU termina de ejecutar la instrucción en curso y **comprueba si hay interrupciones pendientes**

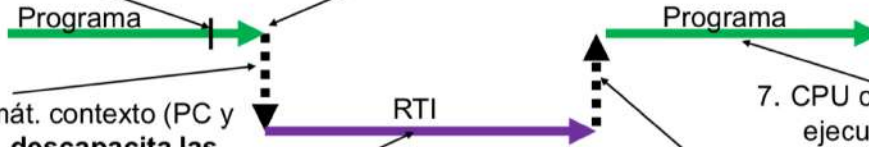
4. CPU salva automat. contexto (PC y reg. de estado), **descapacita las interrupciones y salta a la RTI**

5. CPU ejecuta la RTI, durante la cual:

- Informa al periférico que se ha reconocido su interrupción (por software o por hardware) => El periférico desactiva INTR
- Salva en pila todos los registros de datos y/o direcciones utilizados por la RTI (manual)
- Realiza la operación de E/S con el periférico
- Restaura el contexto arquitectónico salvado
- Realizar el retorno de interrupción correcto

6. CPU capacita las interrupciones, restaura contexto (PC y reg. estado) y retorna al programa

7. CPU continúa con la ejecución del programa

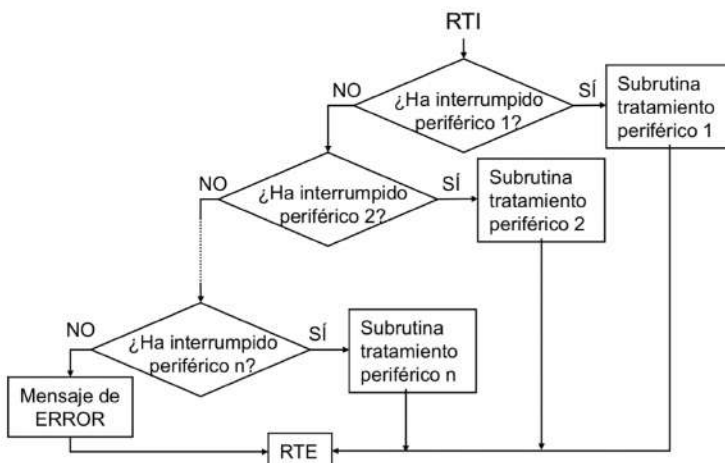


Cuestiones planteadas

- ¿Cómo **se informa al periférico** de que se ha reconocido su interrupción?
Software: borra flag del registro de estado **Hardware**: activa señal de reconocimiento de interrupción
- ¿Cómo **se identifica la fuente de interrupción** y, por tanto, la RTI que hay que ejecutar?
Identificación Software: Polling **Identificación Hardware**: controlador de interrupciones
- ¿Qué ocurre si se produce una segunda interrupción durante la ejecución de la RTI?
⇒ **Interrupciones multinivel y anidamiento de interrupciones**

Pasos para identificar la fuente de interrupción mediante software

- 1) Consulta ordenadamente los registros de control de los controladores HW de los dispositivos mediante Polling. El orden viene determinado por la prioridad de los dispositivos
- 2) Se atiende al primer dispositivo que se encuentre solicitando interrupción → Ejecuta RTI específica
RTI solo borra flag de petición de ese dispositivo



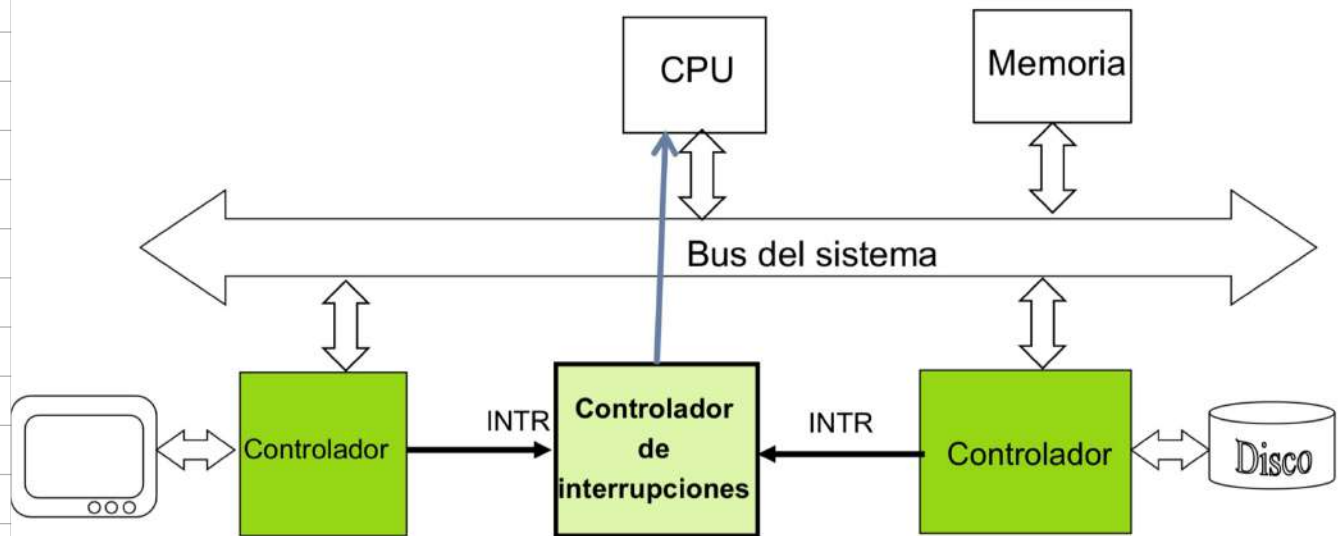
Ventajas

Mecanismo sencillo
Fácil de extender

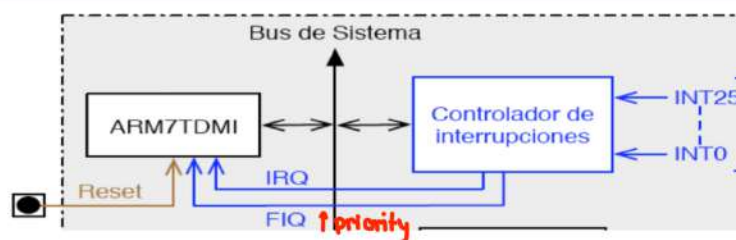
Desventajas

Interfere tiempo en identificar fuente
↳ ralentización
↳ a mayor número de dispositivos, mayor problema

SISTEMA DE E/S CON CONTROLADOR DE INTERRUPCIONES



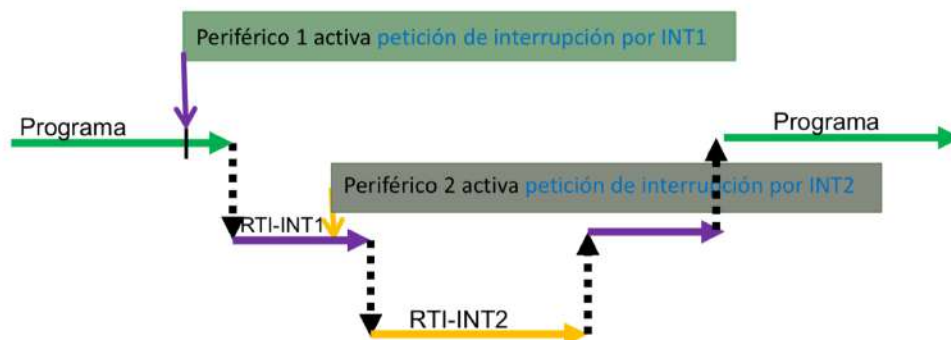
INTERRUPCIONES MULTINIVEL



- ⊙ Varias líneas o niveles de petición de interrupción
 - ⊙ Cada nivel tiene asignada una prioridad distinta
 - ⊙ A cada línea de interrupción se pueden conectar uno o varios dispositivos
- ⊙ Resolución de conflictos de peticiones de interrupción simultáneas
 - ⊙ Peticiones simultáneas por la misma línea
 - Encuesta (software)
 - ⊙ Peticiones simultáneas por líneas distintas
 - Se atiende a la línea más prioritaria



ANIDAMIENTO DE INTERRUPCIONES



ACCESO DIRECTO A MEMORIA (DMA)

Conclusiones

- En el supuesto 2 las interrupciones no ahorran tiempo de CPU frente a la E/S programada porque se ha llegado al límite de velocidad de la línea de interrupción (10 Mbytes/s), por encima de esta velocidad la E/S por interrupción perdería datos
- La E/S programada todavía admitiría más velocidad puesto que no tendría que ejecutar las instrucciones de guarda y restauración del contexto.
- Para dispositivos rápidos se hace necesario un procedimiento de E/S con menor intervención de la CPU

Acceso Directo a Memoria (DMA) transferencia de datos entre un periférico y la memoria sin intervención de la CPU (salvo fase de inicialización de los parámetros de transferencia)

- Con interrupciones se evita bucle de espera pero la transferencia la lleva a cabo el procesador para un bloque de N bytes = N interrupciones
- Con DMA toda la transferencia la realiza el controlador E/S → 1 interrupción final

Es un módulo con capacidad para leer/escribir directamente en la memoria los datos procedentes/emitados de/a los periféricos inicializado por la CPU cuando tiene que realizar una operación de E/S

Pasos:

- 1) solicita a la CPU permiso para acceder a la memoria
- 2) CPU pone en estado de alta impedancia su conexión a los buses del sistema
- 3) cuando DMA finaliza, CPU vuelve a tomar control

La velocidad de transferencia está limitada por el ancho de banda de la memoria

Estructura de un controlador de DMA

- Registro de dirección de memoria almacena dirección inicial de memoria y se incrementa/decrementa después de transferir cada palabra
- Registro de N° palabras almacena el número de palabras a transferir y se decrementa después de transferir cada palabra
- Registro de sentido almacena el sentido de la transferencia (lectura o escritura)

Señales periférico ↔ DMAC

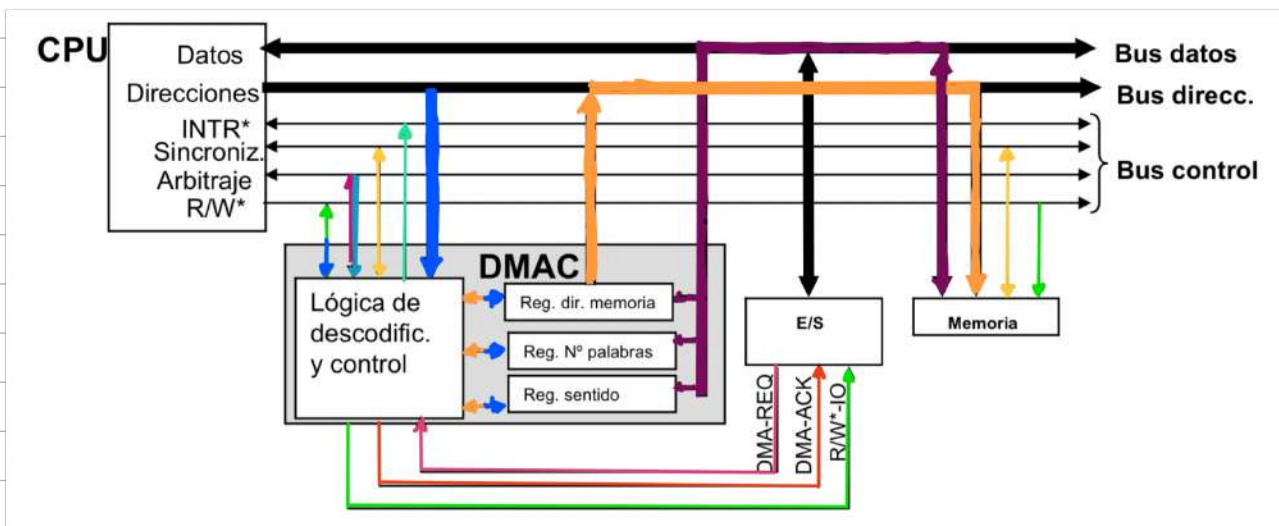
DMA-REQ solicitud de servicio DMA : la activa el periférico para indicar al DMAC que está listo para transmitir/recibir

DMA-ACK concesión del servicio DMA : la activa el DMAC para indicar al periférico que puede realizar la transferencia. El DMAC debe estar en posesión del bus

R/W* - IO sentido de la transferencia para el periférico

Fases de una transferencia

- 1) CPU programa al controlador de DMA para ejecutar la operación de E/S completa
- 2) DMAC programa al controlador de dispositivo para realizar una transferencia como en E/S programada
- 3) El controlador de dispositivo avisa al DMAC de que está listo para realizar la transferencia
- 4) El DMA solicita el control del bus mediante las líneas de arbitraje
- 5) El DMA recibe la concesión del bus e indica al periférico que puede iniciar la transferencia
- 6) El DMA debe generar y procesar las señales del bus adecuadas : dirección de memoria sobre la que se realiza la transferencia, señales de sincronización de la transferencia, señales de lectura/escritura
- 7) Después de la transferencia de cada palabra se actualizan los registros del DMA :
 - Decrementar el registro de N° de palabras
 - Incrementar/decrementar el registro de direcciones en memoria
- 8) se repite hasta que se haya realizado la transferencia de todo el bloque : el controlador del dispositivo avisa al DMAC de que está listo para realizar la transferencia de la siguiente palabra.
- 9) Cuando el registro de n° de palabras llega a cero :
 - DMAC libera el bus y devuelve el control a la CPU
 - DMAC suele activar una señal de interrupción para indicar a la CPU la finalización de la operación E/S solicitada



ACCESO DIRECTO A MEMORIA (DMA)

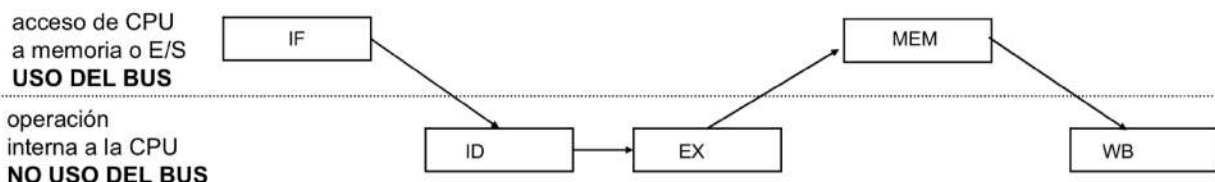
Problema que puede presentar el DMA

- Puede degradar el rendimiento de la CPU si el DMAC hace uso intensivo del bus
 - Si el bus está ocupado en una transferencia DMA, la CPU no puede acceder a memoria para leer instrucciones/datos
- Este problema se reduce con el uso de memoria cache
 - La mayor parte del tiempo, la CPU lee instrucciones de la cache, por lo que no necesita usar el bus de memoria
 - El DMAC puede aprovechar estos intervalos en los que la CPU está leyendo instrucciones de la cache (y por tanto no usa el bus de memoria) para realizar las transferencias
- En caso de computadores sin cache
 - El procesador no utiliza el bus en todas las fases de la ejecución de una instrucción
 - El DMAC puede aprovechar las fases de ejecución de una instrucción en las que la CPU no utiliza el bus para realizar sus transferencias

ACCESO DIRECTO A MEMORIA (DMA)

Problema que puede presentar el DMA

- Si el DMAC sólo toma el control del bus durante los intervalos de tiempo en los que la CPU no hace uso del mismo $\Rightarrow$ *el rendimiento del sistema no sufrirá degradación alguna*



- Se distinguen, por tanto, tres tipos de transferencias:
 - Ráfaga
 - Robo de ciclo
 - Transparente

ACCESO DIRECTO A MEMORIA (DMA)

Control del bus en operaciones de DMA

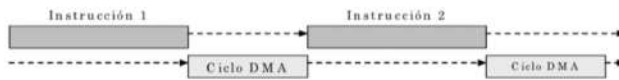
Modo ráfaga

- El DMA toma control del bus durante la transmisión de un bloque de datos completo.
- Consigue alta velocidad de transferencia, dejando inactiva la CPU durante la operación si esta necesita acceder al bus.



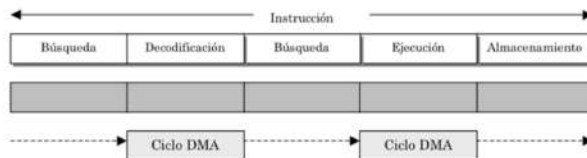
Robo de ciclo

- El DMA toma control del bus y lo retiene durante un solo ciclo para transmitir una palabra.
- Es el modo más usual de transferencia. Se dice que el DMA roba ciclos a la CPU.



Modo transparente

- El DMA accede al bus sólo en los ciclos en los que la CPU no lo utiliza.
- Esto ocurre en diferentes fases de ejecución de las instrucciones, p.e. decodificación
- El programa no se ve afectado en su velocidad de ejecución

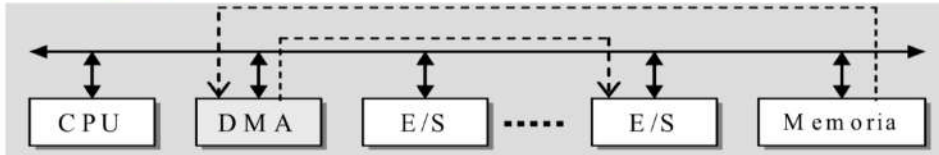


ACCESO DIRECTO A MEMORIA (DMA)

Configuraciones de DMA

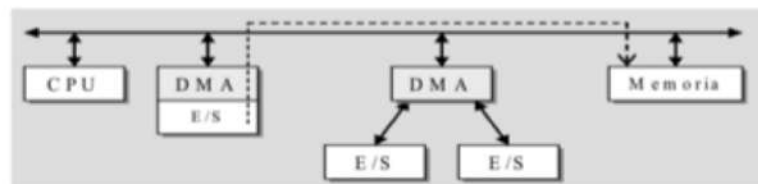
DMA independiente

- Todos los módulos comparten el bus del sistema.
- El DMA hace de intermediario entre la memoria y el periférico.
- Esta configuración es poco eficaz, ya que cada transferencia consume 2 ciclos del bus.



Integración DMA-E/S

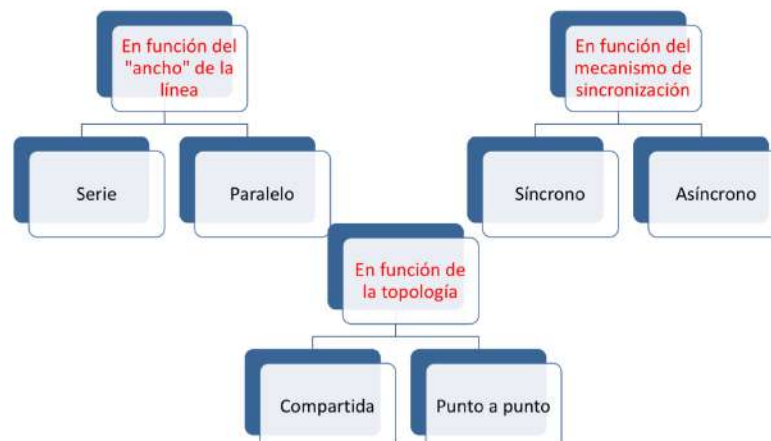
- Proporciona un camino entre el DMA y uno o más controladores de E/S
- Reduce a 1 el número de ciclos de utilización del bus.



Buses de interconexión medio de comunicación para el intercambio de información entre CPU con sistema de memoria y dispositivos de E/S.

Usa de una jerarquía de buses para comunicación a diferentes velocidades de operación

Taxonomía de la líneas de comunicación



Transmisión paralela uso de varias líneas de comunicación para enviar varios bits de información de forma simultánea

- Ancho de banda elevado !!
- Gran coste en la conexión de dispositivos a distancias medias o largas !!
- Frecuencia de funcionamiento limitada por factores físicos !!
- Bajo aprovechamiento del potencial de transmisión paralela por los dispositivos de baja velocidad !!

Transmisión serie uso de una única línea de comunicación (2 cables: dato y tierra) para enviar bits de información de forma secuencial

- Menos costosa !!
- Permite frecuencias de funcionamiento mayores !!
- A igualdad de frecuencia que el caso paralelo, el ancho de banda es menor !!
 - ↳ usar varias conexiones serie para aumentar el ancho de banda

Comunicación

Asíncrona

- Señal de reloj No se envía por línea de comunicación
- Emisor y receptor utilizan sus propios señales de reloj
- Necesidad establecer protocolo sincronización entre ambos
- No necesidad que emisor y receptor funcionen a misma frecuencia

Síncrona

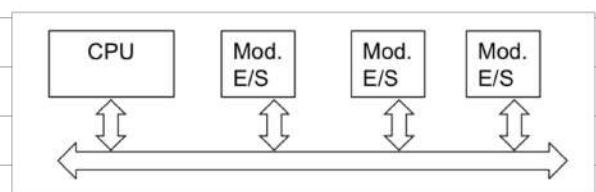
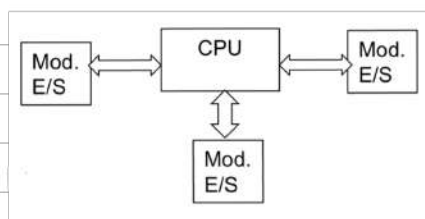
- Señal de reloj viaja con el resto de señales
- Más sencillo y más eficiente
- Necesidad de que emisor y receptor funcionen a la misma frecuencia
- Necesidad de que el emisor sea rápido para que sea rápido

Punto a punto

- Gran rendimiento
- Necesidad gran nº de interfaces / enlaces / pines

Compartida

- Más versátil y barato
- Cuello de botella en comunicación
- Necesidad de arbitraje entre dispositivos para evitar escrituras simultáneas



Organización general de un bus

Líneas de control **implementa** protocolo de comunicación entre componentes

Líneas de datos **transmite** información entre emisor y receptor

Líneas de dirección **contiene** información de direccionamiento (diferentes o multiplexado)

Transferencia de datos en un bus

Master $\xrightarrow{\text{DATO}}$ Slave

Slave $\xrightarrow{\text{DATO}}$ Master

Tipos básicos de transferencia **lectura y escritura**

Fases de una transferencia

- 1) **Identificación del slave**
- 2) **Especificación del tipo de operación**
- 3) **Transferencia del dato**
- 4) **Finalización del ciclo de bus**

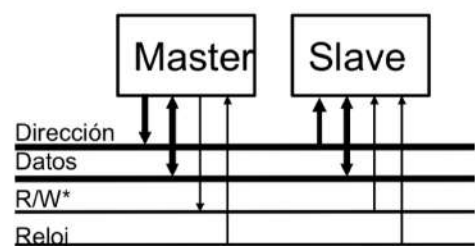
Control de la transferencia

Sincronización: determinar inicio y final de cada transferencia

Arbitraje: control del acceso al bus en caso de varios masters

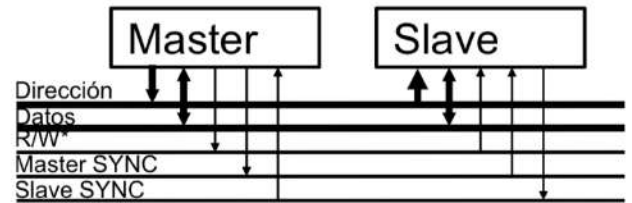
⊙ Sincronización: **protocolo síncrono**

- ⊙ **Existe un reloj que gobierna todas las actividades del bus**, las cuales tienen lugar en un número entero de ciclos de reloj (1 en el ejemplo)
- ⊙ Los **flancos del reloj** (de bajada en el ejemplo) determinan el comienzo de un nuevo ciclo de bus y el final del ciclo anterior
 - ⊙ **Ventajas:**
 - Simplicidad (de diseño y de uso)
 - Sólo se necesita una señal (reloj) para llevar a cabo la sincronización
 - Mayor velocidad (en relación a protocolo asíncrono)
 - ⊙ **Desventajas:**
 - **El periodo de reloj se tiene que adaptar a la velocidad del dispositivo más lento** (por lo que suele usarse para conectar dispositivos homogéneos)
 - No existe confirmación de la recepción de los datos
 - La necesidad de distribuir la señal de reloj limita la longitud del bus



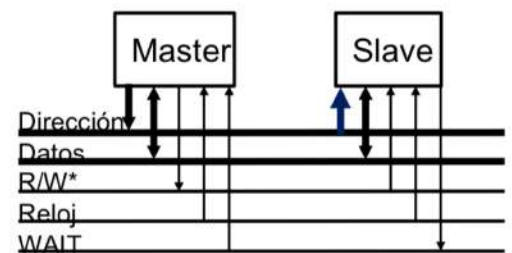
⦿ Sincronización: protocolo asíncrono

- ⦿ No existe señal de reloj
- ⦿ Los dispositivos implicados en la transmisión fijan el comienzo y el final de la misma mediante el intercambio de señales de control (**handshake**)
- ⦿ Se utilizan 2 señales de sincronización:
 - Master SYNC (procedente del master)
 - Slave SYNC (procedente del slave)
- ⦿ Protocolo completamente interbloqueado:
 - A cada flanco de master le sigue uno del slave



⦿ Sincronización: protocolo semisíncrono

- ⦿ Las transferencias se rigen por una única señal de reloj
- ⦿ Cada transferencia puede ocupar uno o varios periodos de reloj
- ⦿ Señal de WAIT. Puede activarla cualquier Slave si no es capaz de realizar la transferencia en 1 solo ciclo. La desactiva cuando es capaz de finalizar con el siguiente flanco de reloj.
- Dispositivos rápidos: operan como en un bus síncrono (una transferencia por ciclo)
- Dispositivos lentos: activan la señal de WAIT y congelan actuación del Master (una transferencia puede ocupar varios ciclos)



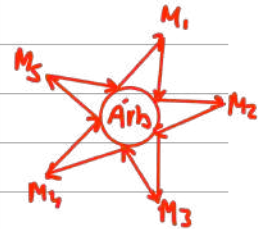
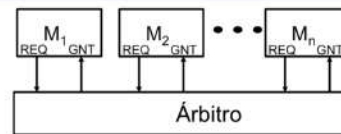
⊙ Conflictos en el acceso: arbitraje

- ⊙ Si puede haber varios emisores (Masters), es necesario que el acceso al bus sea *libre de conflictos*
 - ⊙ Se debe evitar que dos módulos escriban simultáneamente en el bus
 - ⊙ Cada dispositivo solicita permiso para poder tomar el control del bus
- ⊙ **Protocolos centralizados:** existe un árbitro del bus o máster principal que controla el acceso al bus
 - ⊙ En estrella
 - ⊙ Daisy-chain de 3 y 4 hilos
- ⊙ **Protocolos distribuidos:** el control de acceso al bus se lleva a cabo entre todos los posibles masters de una forma cooperante
 - ⊙ Protocolo de líneas de identificación

Centralizados

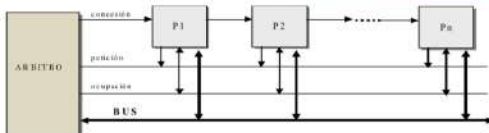
⊙ Protocolos centralizados: en estrella

- ⊙ Cada máster se conecta al árbitro mediante dos líneas individuales:
 - ⊙ BUS REQUEST (REQ): línea de petición del bus
 - ⊙ BUS GRANT (GNT): línea de concesión del bus
- ⊙ Varias peticiones de bus pendientes: el árbitro puede aplicar distintos algoritmos de decisión:
 - ⊙ FIFO, Prioridad fija, Prioridad variable
- ⊙ **Ventajas:**
 - ⊙ Algoritmos de arbitraje simples
 - ⊙ Pocos retardos de propagación de las señales (en comparación con daisy-chain)
- ⊙ **Desventajas:**
 - ⊙ Número elevado de líneas de arbitraje en el bus (dos por cada posible máster)
 - ⊙ Número de másteres alternativos limitado por el número de líneas de arbitraje
- ⊙ Ejemplo: PCI



⊙ Protocolo de encadenamiento (daisy chaining) de 3 señales

- ⊙ Utiliza tres líneas para gestionar el arbitraje: petición, ocupación y concesión.
 - ⊙ La línea de petición es compartida por todos los procesadores a través de una entrada al árbitro con capacidad de O-cableada.
 - ⊙ El árbitro activa la concesión cuando recibe una petición y la de ocupación está desactivada.
 - ⊙ Cuando un procesador toma el control del bus activa la línea de ocupación.
 - ⊙ Si un procesador recibe la concesión y no ha solicitado el bus, transmite la señal al siguiente.
 - ⊙ Un procesador toma el control del bus si tiene una petición local pendiente, la línea de ocupación está desactivada y recibe el flanco de subida de la señal de concesión.



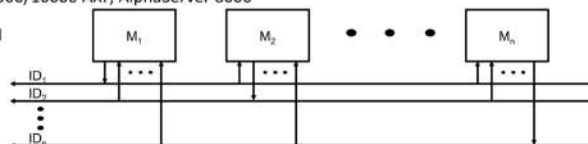
⊙ Protocolo de encadenamiento (daisy chaining) de 4 señales

- ⊙ Permite simultanear el uso del bus por un procesador con el arbitraje para seleccionar el procesador que utilizará el bus en la siguiente transacción.
- ⊙ Cuando el primer procesador libera el bus, el arbitraje para el siguiente ya se ha realizado.
- ⊙ La línea nueva de **reconocimiento** la activa un procesador que solicitó el bus (activo petición) y recibió la concesión pero la línea de ocupación estaba activa (bus ocupado).
- ⊙ Cuando el árbitro recibe la activación de reconocimiento inhibe su actuación, es decir, deja de atender la señal de petición y generar la de concesión.
- ⊙ El procesador queda en espera de ocupar el bus cuando lo abandone su actual usuario desactivando la señal ocupación.
- ⊙ Cuando esto ocurre, el procesador ocupa el bus y desactiva la señal de reconocimiento.

Distribuidos

⊙ Protocolo de líneas de identificación

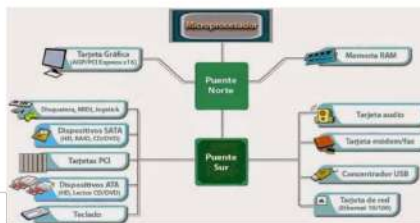
- ⊙ Cuando un máster quiere tomar el control del bus activa su línea de identificación
- ⊙ Cada línea de identificación tiene asignada una prioridad: $(ID_1) < (ID_2) < \dots < (ID_n)$
- ⊙ Si varios masters activan simultáneamente sus líneas de identificación, gana el de mayor prioridad
- ⊙ Funcionamiento alternativo: las prioridades pueden ser variables
- ⊙ **Desventajas:**
 - ⊙ Número de dispositivos limitado por el número de líneas de arbitraje
- ⊙ **Prioridad variable:** DEC 7000/10000 AXP, AlphaServer 8000
- ⊙ **Prioridad fija:** VAX SBI, SCSI



⊙ Puentes de comunicación

- ⊙ El Puente norte controla la conexión de la CPU con los componentes de alta velocidad del computador:
 - Memoria RAM
 - Buses de alta velocidad
 - Puente sur
- ⊙ El Puente sur se encarga de coordinar los diferentes dispositivos de entrada/salida y algunas otras funcionalidades de baja velocidad.
- ⊙ Se comunica indirectamente con la CPU a través del puente norte.
- ⊙ Un Puente sur actual puede incluir soporte para:

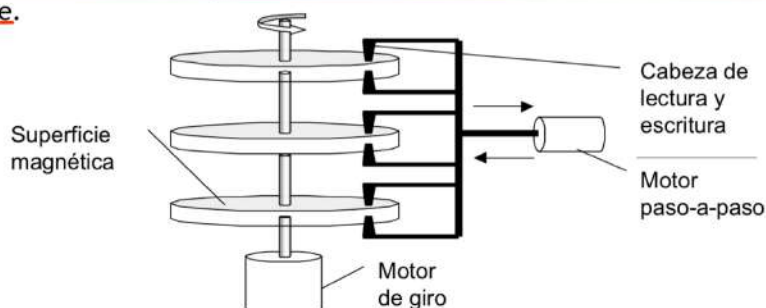
- Bus PCI
- Bus ISA
- Controladores de interrupción
- Ethernet
- RAID
- USB
- Codec de Audio



Dispositivos de almacenamiento

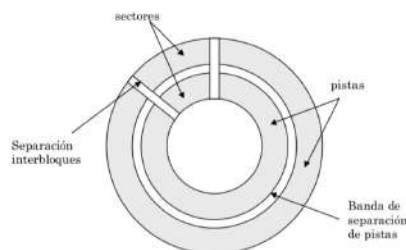
⊙ Discos magnéticos

- ⊙ Son memorias secundarias que se conectan al computador como dispositivos periféricos.
- ⊙ Forman parte de la jerarquía de memoria del computador.
- ⊙ Están constituidos por superficies circulares recubiertas por un material ferromagnético.
- ⊙ La información se asocia al sentido de la magnetización de pequeñas áreas de su superficie.
- ⊙ El conjunto de superficies gira a una velocidad constante por la acción de un motor.
- ⊙ La información se escribe y lee a través de un conjunto de cabezas que se mueve radialmente.



⊙ Formato de grabación

- ⊙ Los datos se organizan en un conjunto de anillos concéntricos denominados *pistas*.
- ⊙ Las pistas adyacentes están separadas por bandas vacías.
- ⊙ Para simplificar la electrónica, se suele almacenar el mismo número de bits en cada pista.
- ⊙ Los datos se transfieren en bloques y se almacenan en regiones denominadas *sectores*.
- ⊙ Suele haber entre 10 y 100 sectores por pista, y estos pueden ser de longitud fija o variable.
- ⊙ Los sectores adyacentes se separan con regiones vacías.
- ⊙ Se graban algunos datos extra utilizados sólo por el controlador del disco (no accesibles al usuario).

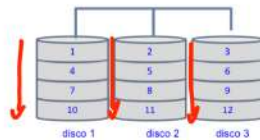


RAID (Redundant Array of Independent/Inexpensive Disks)

- Es un método de combinación de dos o más discos para formar una única unidad lógica
- Utilizan dos principios básicos: la redundancia y el paralelismo
- Aumentando el grado de redundancia se consigue mayor fiabilidad (tolerancia de fallos)
- Grabando un bloque de datos entre varios discos se consigue acelerar su acceso (paralelismo)
- Existen diferentes opciones denominadas niveles de RAID
- Cada nivel proporciona un equilibrio distinto entre tolerancia a fallos, rendimiento y coste
- Cada nivel de RAID se ajusta mejor a determinadas aplicaciones y entornos
- Existen siete niveles de RAID (0 al 6) definidos y aprobados por el RAID Advisory Board

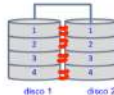
RAID Nivel 0

- Distribuye los datos a través de varios discos y no proporciona redundancia
- Maneja acceso simultáneo a varios discos, lo que proporciona mayor velocidad de acceso
- No es tolerante a fallos: si un disco falla el sistema falla
- Si un archivo ocupa las divisiones 1, 2 y 3 (ver dibujo) se puede recuperar en la tercera parte del tiempo que se tardaría si las tres divisiones estuvieran en el mismo disco. *mejor*
- Se utiliza en tratamiento de imagen, sonido, vídeo y en general en aplicaciones que requieran gran velocidad de acceso y puedan tolerar fallos



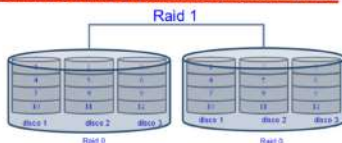
RAID Nivel 1

- Utiliza una copia espejo (mirroring) para proporcionar redundancia tolerante a fallos
- Los discos guardan exactamente la misma información por parejas
- Cuando un disco falla, su espejo puede recuperarlo, pero aumenta el coste del sistema



RAID 0 - 1

- Cada conjunto RAID 0 posee una copia espejo (RAID 1)
- Incorpora a la alta velocidad del RAID 0 la tolerancia de fallos de RAID 1
- Coste elevado



RAID 1 - 0

- Conjuntos RAID 1 (con su redundancia y tolerancia fallos) se organizan como un RAID 0



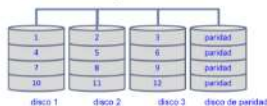
RAID Nivel 2

- Utiliza códigos de corrección de Hamming.
- Realiza acceso paralelo y sincronizado a todos los discos, datos y corrección de errores (ECC)



RAID Nivel 3

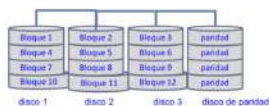
- Distribuye los datos por múltiples discos a nivel de bytes e introduce el chequeo de paridad
- Añade redundancia utilizando un disco de paridad que permite recuperar errores producidos por un fallo en uno cualquier disco
- Si el fallo se produce en el disco de paridad, se pierde la redundancia, pero se mantiene intacta la información original.



La paridad se calcula con la función O-exclusiva:
Si se produce un error en el disco 1 se puede recuperar su contenido con el contenido de los otros dos y el de paridad:
Análogamente para los discos 2 y 3:

RAID Nivel 4

- Distribuye los datos por bloques entre varios discos, con la paridad en un disco.
- La paridad permite la recuperación de cualquier disco en caso de fallo.
- El rendimiento es muy bueno para lecturas (similar al nivel 0).
- Penaliza las escrituras porque requiere actualizar siempre los datos de paridad



RAID Nivel 5

- Crea datos de paridad, distribuyéndolos a través de todos los discos, excepto en aquel disco en que se almacena la información original. Es la alternativa más popular.
- Es el más completo de todos los niveles de redundancia, si un disco falla la información de paridad en los otros permite la reconstrucción del fallo.
- Escribe datos en los discos al nivel de bloques, siendo más apropiado para múltiples transacciones pequeñas como e-mail, procesadores de palabras, hojas electrónicas, etc.

